

BHARATBAZZAR HANDCRAFT & ARTISAN E-COMMERCE SYSTEM

A Project Report

Submitted in partial fulfilment for the requirements of
DBMS (CSE209) Project

Submitted by

AMAN MADDHESHIYA

Roll No.: AP24110011314

Under the Supervision of

Dr. Prasanthi Boyapati

Assistant Professor

Department of Computer Science and Engineering
SRM University-AP

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRM UNIVERSITY-AP

NEERUKONDA, MANAGALAGIRI - 522503

ANDHRA PRADESH, INDIA

APRIL 2026

CERTIFICATE

This is to certify that the project work entitled "**BharatBazaar Handcraft & Artisan E-Commerce System**" is a Bonafide record of project work carried out by the following student:

Mr. Aman Maddheshiya (Roll No.: AP24110011314)

from the Department of Computer Science and Engineering, **SRM University-AP**. The student conducted this project work under my supervision during the period **January 2026 to April 2026**.

It is further certified that, to the best of my knowledge, this project has not previously formed the basis for the award of any degree or any similar title to this or any other candidate. This is also to certify that the project work represents the **individual effort** of the candidate.

Station: Mangalagiri

Date: April 2026

Dr. Prasanthi Boyapati

Assistant Professor

Department of Computer Science & Engineering

SRM University-AP, Andhra Pradesh.

ABSTRACT

BharatBazaar is a full-stack, database-driven e-commerce platform purpose-built for handcraft and artisan products. The system connects skilled artisans from across India with buyers who value authentic, hand-made goods such as pottery, handwoven textiles, natural skincare products, woodcraft, candles, and macrame art. The platform is developed as a DBMS course project that demonstrates the application of relational database design principles in a real-world, production-quality scenario.

The backend is powered by a normalized MySQL 8.0 database that incorporates advanced SQL constructs including stored procedures, triggers, and views. The database schema is designed to Third Normal Form (3NF), ensuring minimal data redundancy and maximum integrity. Key database objects include the PlaceOrder and ProcessPayment stored procedures for transactional order management, three triggers for automatic inventory deduction and order confirmation, and two views for aggregated reporting of top-selling products and user activity summaries.

The backend API is built with Node.js and Express 4, exposing a RESTful interface secured with JWT-based authentication and bcrypt password hashing. The frontend is a responsive, multi-page application built entirely with vanilla HTML5, CSS3, and JavaScript, featuring an earthy and artisan-themed design system. Key user-facing features include product search and filtering by category, price range, and sorting criteria; a shopping cart with quantity management; a full checkout flow with address management, payment method selection, and coupon code application; and an order history page with real-time status tracking.

The project demonstrates a complete software development lifecycle from requirements gathering and entity-relationship modelling through relational schema design, backend API development, and frontend implementation, making it a comprehensive reference implementation for database-centric web application development.

TABLE OF CONTENTS

Section	Page
1. Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	6
2. Requirement Analysis	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	8
3. Conceptual Models	10
3.1 Entity-Relationship (ER) Diagram	10
3.2 Relational Schema	11
4. System Design	14
4.1 Design Goals	14
4.2 System Architecture & Modules	15
4.3 Detailed Design (Tables & Relationships)	17
5. Conclusion	22

1. INTRODUCTION

1.1 Background

India has a centuries-old tradition of handcraft and artisanal production, with millions of skilled craftspeople working across disciplines such as pottery, weaving, woodwork, natural cosmetics, and fibre arts. Despite the quality and cultural richness of their work, these artisans have historically been confined to local markets or fragmented supply chains that limit their income and reach. The rapid expansion of e-commerce over the past decade has created an unprecedented opportunity to connect these artisans directly with urban and global consumers who actively seek authentic, sustainable, and ethically produced goods.

However, most mainstream e-commerce platforms are designed around mass-produced goods and do not provide the storytelling, category granularity, or artisan-brand identity that handcraft products require. BharatBazaar is designed specifically for this gap: a platform that treats every product as a unique handcrafted item with its own provenance, brand identity, and inventory reality. The database and application architecture of BharatBazaar is designed from first principles to support this domain.

This project also serves as a comprehensive academic exercise in relational database management systems. It applies normalisation theory, transactional integrity, trigger-based automation, and stored-procedure-driven business logic in a context rich enough to demonstrate all these concepts meaningfully.

1.2 Problem Statement

The handcraft and artisan product market in India faces several structural problems that a well-designed e-commerce system can address:

- **Fragmented inventory management:** Artisans produce in small batches, and stock levels change rapidly. A system must track inventory separately from product definitions and decrement it atomically at the point of order placement to prevent overselling.
- **Price consistency across order history:** Product prices change over time, but a historical order must always reflect the price at the time of purchase. This requires a price-snapshot architecture in order line items.

- Complex discount and coupon logic: Promotional codes with different discount types (percentage vs. flat), minimum order thresholds, and usage limits must be evaluated correctly at checkout without allowing invalid applications.
- Transactional integrity during checkout: The act of placing an order involves multiple database writes (creating an order, inserting order items, deducting inventory, clearing the cart) that must all succeed atomically or all roll back, ensuring data consistency.
- Lack of a curated, artisan-first browsing experience: Consumers searching for handcrafted goods need category hierarchies tailored to artisan product types, not generic retail taxonomies.

BharatBazzar addresses all of these problems through a carefully designed relational schema, stored procedures that encapsulate transactional business logic, and triggers that automate derived state changes such as inventory deduction and order confirmation.

1.3 Objectives of the Project

The primary objectives of the BharatBazzar project are as follows:

- Design a fully normalised (up to 3NF) relational database schema that accurately models all entities and relationships in an artisan e-commerce domain.
- Implement advanced SQL database objects including stored procedures for order placement and payment processing, triggers for inventory deduction and order status management, and views for analytical reporting.
- Build a secure, stateless RESTful API using Node.js and Express, protected by JWT authentication and bcrypt password hashing, that exposes all required business operations.
- Develop a responsive, multi-page frontend application using vanilla HTML, CSS, and JavaScript with a custom design system themed around earthy and artisan aesthetics.
- Demonstrate end-to-end transactional integrity: from product browsing through cart management, checkout with coupon validation, payment simulation, and order history display.

- Provide a practical reference implementation of all major DBMS course concepts: normalisation, keys, foreign key constraints, transactions, triggers, stored procedures, views, and indexing.

2. REQUIREMENT ANALYSIS

2.1 Functional Requirements

The following functional requirements were identified through analysis of the e-commerce domain and the specific needs of an artisan marketplace:

2.1.1 User Authentication & Account Management

- The system shall allow new users to register by providing their full name, email address, phone number, and password. The password shall be stored as a bcrypt hash.
- The system shall authenticate registered users via email and password, issuing a JWT access token upon successful login.
- The system shall support two user roles: customer and admin. Customers have access to the shopping experience; admins have access to management endpoints.
- A logged-in user shall be able to add, view, and manage multiple shipping addresses, designating one as their default.

2.1.2 Product Browsing & Discovery

- The system shall display a paginated product catalogue sourced from the database, including product name, brand, category, price, stock level, and average customer rating.
- Users shall be able to filter products by category (including subcategories), minimum and maximum price, and sort results by price (ascending/descending), average rating, or creation date.
- Users shall be able to perform a text-based search across product names and descriptions.

- Products with zero inventory shall be visually flagged as out of stock and the add-to-cart button shall be disabled.

2.1.3 Shopping Cart Management

- Each authenticated user shall have exactly one active shopping cart, automatically created upon registration.
- Users shall be able to add products to their cart, specifying a quantity. Adding a product already in the cart shall increment the quantity.
- Users shall be able to update the quantity of any cart item or remove it entirely from the cart.
- Users shall be able to clear all items from their cart in a single operation.
- The cart summary shall display a real-time subtotal calculated from current product prices and quantities.

2.1.4 Checkout & Order Placement

- The system shall allow users to select an existing delivery address or add a new one during checkout.
- Users shall be able to select a payment method from: UPI, credit card, debit card, net banking, wallet, or cash on delivery.
- Users shall be able to apply an optional coupon code. The system shall validate the code against active coupons in the database, checking type (percent/flat), minimum order amount, and active status before applying the discount.
- Order placement shall be executed as an atomic database transaction via the PlaceOrder stored procedure, which creates the order record, inserts line items with price snapshots, and clears the cart.
- Payment processing shall be executed via the ProcessPayment stored procedure, which generates a unique transaction reference and marks the order as confirmed.
- The system shall prevent order placement if insufficient inventory exists for any item in the cart (enforced by a BEFORE INSERT trigger on Order_Items).

2.1.5 Order History & Tracking

- Authenticated users shall be able to view all their past orders, including order ID, date, item list with quantities and line totals, delivery address, payment method, transaction reference, discount applied, and total amount.
- Each order shall display its current status: pending, confirmed, shipped, delivered, or cancelled.

2.1.6 Reviews & Wishlist

- Authenticated users shall be able to submit one review per product, including a 1-5 star rating, a title, and a detailed body.
- Products shall display aggregated average ratings and review counts sourced from the Reviews table.
- Users shall be able to maintain a personal wishlist of products (many-to-many relationship).

2.2 Non-Functional Requirements

2.2.1 Performance

The product listing API endpoint shall respond within 500 milliseconds under normal load conditions. Query performance is optimised through foreign key indexes created automatically by MySQL on all FK columns. The Inventory table is kept separate from Products (3NF) to allow high-frequency inventory updates without locking the main product table. The database uses a promise-based connection pool (mysql2) to reuse connections rather than opening a new connection per request.

2.2.2 Security

All user passwords are stored exclusively as bcrypt hashes with a cost factor of 10; plaintext passwords are never persisted anywhere in the system. All authenticated API routes are protected by a JWT middleware layer that validates the token signature and expiry on every request. The database does not expose any direct query construction from user input; all parameterised queries are passed through the mysql2 driver's prepared statement interface, preventing SQL injection. Tokens are invalidated client-side on logout by clearing localStorage.

2.2.3 Usability

The frontend is fully responsive, adapting to screen widths down to 360px. The design system employs a consistent earthy colour palette, readable typography (Outfit and Lora from Google Fonts), clear visual hierarchy, and contextual feedback via a toast notification system. Interactive elements such as quantity controls, payment method selection, and address cards provide immediate visual feedback. Loading states (spinner overlay) and empty states (illustrated empty cart/order screens) are implemented throughout to prevent user confusion.

2.2.4 Reliability and Availability

Database transactions in the PlaceOrder and ProcessPayment stored procedures are wrapped in START TRANSACTION / COMMIT / ROLLBACK blocks with DECLARE EXIT HANDLER for SQLEXCEPTION clauses, ensuring that any partial failure rolls back all changes atomically. The BeforeOrderItemsInsert trigger enforces stock validation at the database layer, ensuring that inventory constraints are upheld regardless of which application layer issues the insert. The API handles 401 Unauthorized responses globally, redirecting expired-session users to the login page automatically.

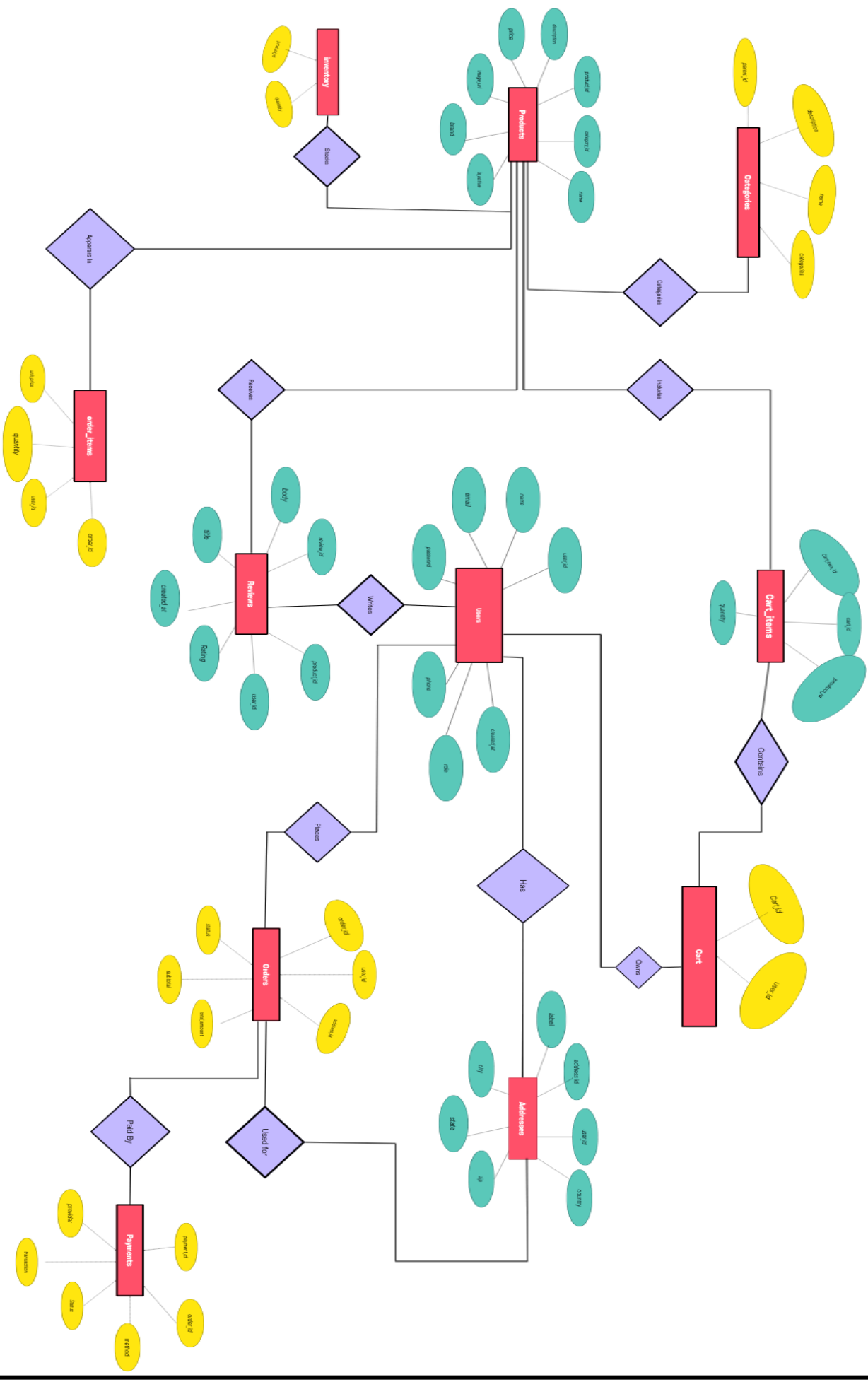
3. CONCEPTUAL MODELS

3.1 Entity-Relationship (ER) Diagram

The ER diagram for BharatBazaar identifies twelve primary entities and the relationships between them. The following section describes each entity, its key attributes, and its cardinality relationships with other entities.

3.1.1 Entities and Their Attributes

Entity	Primary Key	Key Attributes
Users	user_id (INT, AI)	name, email (UNIQUE), password, phone, role (ENUM)
Addresses	address_id (INT, AI)	user_id (FK), label, street, city, state, zip, country, is_default
Categories	category_id (INT, AI)	name, description, parent_id (FK, self-ref)
Products	product_id (INT, AI)	category_id (FK), name, description, price, image_url, brand, is_active
Inventory	product_id (FK, PK)	quantity (INT)
Cart	cart_id (INT, AI)	user_id (FK, UNIQUE)
Cart_Items	cart_item_id (INT, AI)	cart_id (FK), product_id (FK), quantity — UNIQUE(cart_id, product_id)
Orders	order_id (INT, AI)	user_id (FK), address_id (FK), status (ENUM), subtotal, discount, total_amount
Order_Items	order_item_id (INT, AI)	order_id (FK), product_id (FK), quantity, unit_price (snapshot)
Payments	payment_id (INT, AI)	order_id (FK, UNIQUE), method, status (ENUM), transaction_ref, provider, paid_at
Reviews	review_id (INT, AI)	product_id (FK), user_id (FK), rating (1-5), title, body — UNIQUE(user_id, product_id)
Coupons	coupon_id (INT, AI)	code (UNIQUE), discount_type (ENUM), discount_value, min_order_amt, is_active
Wishlist	(user_id, product_id) composite PK	user_id (FK), product_id (FK)



3.2 Relational Schema

The relational schema is derived directly from the ER diagram. Each entity becomes a table, and each relationship is represented by foreign key constraints. The following schemas define each table with its complete attribute list, types, and constraints.

Users Table

Attribute	Data Type	Constraints
user_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL (bcrypt hash)
phone	VARCHAR(20)	NULLABLE
role	ENUM('customer','admin')	DEFAULT 'customer'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Addresses Table

Attribute	Data Type	Constraints
address_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FK -> Users.user_id, ON DELETE CASCADE
label	VARCHAR(50)	DEFAULT 'Home'
street	VARCHAR(255)	NOT NULL
city	VARCHAR(100)	NOT NULL
state	VARCHAR(100)	NOT NULL
zip	VARCHAR(20)	NOT NULL
country	VARCHAR(100)	DEFAULT 'India'
is_default	BOOLEAN	DEFAULT FALSE

Products Table

Attribute	Data Type	Constraints
-----------	-----------	-------------

product_id	INT	PRIMARY KEY, AUTO_INCREMENT
category_id	INT	FK -> Categories.category_id, ON DELETE SET NULL
name	VARCHAR(255)	NOT NULL
description	TEXT	NULLABLE
price	DECIMAL(10,2)	NOT NULL
image_url	VARCHAR(500)	NULLABLE
brand	VARCHAR(100)	NULLABLE
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Orders Table

Attribute	Data Type	Constraints
order_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FK -> Users.user_id
address_id	INT	FK -> Addresses.address_id, ON DELETE SET NULL
status	ENUM	'pending','confirmed','shipped','delivered','cancelled'
subtotal	DECIMAL(10,2)	NOT NULL
discount	DECIMAL(10,2)	DEFAULT 0.00
total_amount	DECIMAL(10,2)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Order_Items Table (Price Snapshot)

Attribute	Data Type	Constraints
order_item_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FK -> Orders.order_id, ON DELETE CASCADE
product_id	INT	FK -> Products.product_id
quantity	INT	NOT NULL
unit_price	DECIMAL(10,2)	NOT NULL — price at time of purchase

Payments Table

Attribute	Data Type	Constraints
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FK -> Orders.order_id, UNIQUE, ON DELETE CASCADE
method	VARCHAR(50)	NOT NULL (e.g., 'upi', 'credit_card')
status	ENUM	'pending','completed','failed','refunded' — DEFAULT 'pending'
transaction_ref	VARCHAR(255)	Generated UUID-based reference
provider	VARCHAR(100)	Payment provider name
paid_at	TIMESTAMP	NULL until payment completed

4. SYSTEM DESIGN

4.1 Design Goals

The system design of BharatBazzar was guided by the following primary design goals:

- **Normalisation to 3NF:** Every table is designed to be in Third Normal Form. Transitive dependencies are eliminated; for example, inventory quantity is not stored in the Products table but in a separate Inventory table. This prevents anomalies when updating stock levels and allows inventory to be updated without locking the broader product record.
- **Transactional integrity:** All multi-step write operations (order placement, payment processing) are wrapped in explicit database transactions. Partial failures must trigger a complete rollback, ensuring the database is never left in an inconsistent intermediate state.
- **Price immutability in historical orders:** The unit_price column in Order_Items stores the product price at the time of purchase as an immutable snapshot. This means that subsequent price changes to a product do not retroactively alter the value of historical orders.

- Database-enforced business rules: Critical constraints such as preventing orders that exceed available inventory are enforced at the database layer via BEFORE INSERT triggers on Order_Items, not just in application code. This ensures integrity regardless of which application or tool accesses the database.
- Separation of concerns: The system is divided into three distinct layers: a MySQL database layer, a Node.js/Express API layer, and a vanilla JS frontend layer. Each layer has well-defined responsibilities and communicates through documented interfaces (SQL stored procedures for the DB-API boundary; RESTful JSON endpoints for the API-frontend boundary).
- Scalable category hierarchy: The Categories table uses a self-referencing parent_id foreign key, supporting a two-level hierarchy (parent categories and subcategories) without requiring schema changes to add deeper nesting in the future.

4.2 System Architecture & Modules

4.2.1 Three-Tier Architecture

BharatBazzar follows a classical three-tier client-server architecture:

- Presentation Tier (Frontend): Five HTML pages (index.html, products.html, cart.html, checkout.html, orders.html) together with a shared CSS design system (style.css) and a centralised JavaScript API client (api.js). All pages are statically served and communicate with the backend exclusively via fetch() calls to the REST API.
- Logic Tier (Backend API): A Node.js application using the Express 4 framework, running on port 5001. Routes are organised by domain: auth, products, cart, orders, and admin. Controllers contain the business logic, including calls to stored procedures and parameterised SQL queries via the mysql2 connection pool.
- Data Tier (Database): A MySQL 8.0 database named shopdb containing 12 tables, 2 views, 2 stored procedures, and 3 triggers. The database encapsulates all persistence logic, constraint enforcement, and transactional integrity.

4.2.2 Backend Module Breakdown

Module / Route	Responsibility
----------------	----------------

/api/auth	User registration (POST /register), login (POST /login), profile retrieval (GET /me). Issues and validates JWT tokens.
/api/products	Product listing with dynamic filters (category, price range, search, sort). Single product detail. Category listing.
/api/cart	GET cart with computed line totals, POST add item, PUT update quantity, DELETE remove item, DELETE clear all.
/api/orders	POST place order (calls PlaceOrder SP + ProcessPayment SP), GET order history with full join, GET/POST addresses.
/api/admin	GET dashboard statistics, GET top products (via vw_top_products view), GET user summaries (via vw_user_summary view), GET all orders.
Middleware: auth.js	JWT verification middleware applied to all protected routes. Decodes token, attaches user object to req.user.
Config: db.js	MySQL2 connection pool configuration. Reads credentials from .env. Exports a promise-based pool instance.

4.2.3 API Endpoint Summary

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register new user
POST	/api/auth/login	No	Login and receive JWT
GET	/api/auth/me	Yes	Get current user profile
GET	/api/products	No	List products with filters/search/sort
GET	/api/products/:id	No	Get single product details
GET	/api/products/categories	No	List all categories
GET	/api/cart	Yes	Get cart with items and subtotal
POST	/api/cart/add	Yes	Add item to cart
PUT	/api/cart/update	Yes	Update item quantity
DELETE	/api/cart/remove	Yes	Remove single item
DELETE	/api/cart/clear	Yes	Clear entire cart
POST	/api/orders/place	Yes	Place order (calls stored procedures)
GET	/api/orders	Yes	Get order history with full details
GET	/api/orders/addresses	Yes	List saved addresses
POST	/api/orders/addresses	Yes	Add new delivery address

GET	/api/admin/stats	Admin	Dashboard statistics
GET	/api/admin/top-products	Admin	Top products via vw_top_products
GET	/api/admin/users	Admin	User summaries via vw_user_summary

4.3 Detailed Design

4.3.1 Stored Procedures

Two stored procedures encapsulate the core transactional business logic of the application:

PlaceOrder(p_user_id, p_address_id, p_coupon_code, OUT p_order_id, OUT p_msg)

This procedure manages the complete order placement workflow within a single database transaction. It first retrieves the user's cart ID and computes the subtotal by joining Cart_Items with Products. If a coupon code is provided, it validates the code against the Coupons table, checking is_active status and min_order_amt threshold, then applies either a percentage or flat discount. It inserts the order header into Orders with the computed subtotal, discount, and total, captures the generated order_id via LAST_INSERT_ID(), then inserts all cart line items into Order_Items as price snapshots. The AFTER INSERT trigger on Order_Items automatically decrements Inventory. Finally, the procedure deletes all Cart_Items for the cart. Any SQL exception triggers a ROLLBACK, and the OUT parameters return error information to the caller.

ProcessPayment(p_order_id, p_method, p_provider, OUT p_pay_id, OUT p_msg)

This procedure handles payment recording within a transaction. It generates a unique transaction reference using CONCAT('TXN-', UUID()), inserts a completed payment record into the Payments table, updates the order status to 'confirmed', and returns the generated payment_id and a success message. The AfterPaymentComplete trigger provides an additional safety net that also confirms the order whenever a payment status changes to 'completed'.

4.3.2 Triggers

Trigger Name	Timing & Event	Action
BeforeOrderItemsInsert	BEFORE INSERT on Order_Items	Reads current stock from Inventory. If stock < requested quantity, raises SQLSTATE '45000' with message 'Insufficient stock for product', aborting the insert and rolling back the transaction.
AfterOrderItemsInsert	AFTER INSERT on Order_Items	Decrements Inventory.quantity by NEW.quantity for the inserted product_id. This is the primary mechanism for stock deduction.
AfterPaymentComplete	AFTER UPDATE on Payments	Checks if NEW.status = 'completed' AND OLD.status != 'completed'. If true, updates the associated Orders record status to 'confirmed'. Acts as a safety net independent of the stored procedure.

4.3.3 Views

View Name	Definition and Purpose
vw_top_products	Joins Products with Order_Items and groups by product_id to compute total units sold (SUM(quantity)). Orders results descending by total_sold. Used by the admin dashboard to surface best-selling artisan products.
vw_user_summary	Left joins Users with Orders and groups by user_id to compute total orders placed (COUNT) and total spend (SUM of total_amount, COALESCE to 0 for users with no orders). Used by the admin user management endpoint.

4.3.4 System Flow

The complete end-to-end order flow proceeds as follows:

- Step 1 — Authentication: The user registers or logs in via `/api/auth`. A JWT token is issued and stored in the browser's `localStorage`. All subsequent API requests include this token in the Authorization header.
- Step 2 — Product Discovery: The user browses `/api/products` with optional query parameters for category, search term, price range, and sort order. The API builds a dynamic SQL query and returns product data with aggregated ratings from the Reviews table and stock levels from Inventory.
- Step 3 — Cart Management: The user adds items via `POST /api/cart/add`. The API upserts into `Cart_Items` (`INSERT ON DUPLICATE KEY UPDATE` for MySQL). Quantities can be adjusted or items removed. The cart subtotal is

computed dynamically on every GET /api/cart call by joining Cart_Items with Products.

- Step 4 — Checkout Initiation: The frontend fetches the cart contents and the user's saved addresses. The user selects an address, payment method, and optionally enters a coupon code.
- Step 5 — Order Placement: The frontend calls POST /api/orders/place with the address ID, coupon code, payment method, and provider. The API executes CALL PlaceOrder() followed by CALL ProcessPayment(). Inventory is decremented by the BeforeOrderItemsInsert/AfterOrderItemsInsert trigger pair. The cart is cleared atomically within the stored procedure.
- Step 6 — Order Confirmation: The success response includes the order ID and transaction reference, displayed on a confirmation screen. The cart badge in the navbar resets to zero.
- Step 7 — Order History: The user views /api/orders, which returns all their orders with a full JOIN across Orders, Order_Items, Products, Addresses, and Payments tables, providing complete order detail in a single query.

4.3.5 Frontend Design System

The frontend implements a comprehensive design system defined in style.css using CSS custom properties (variables). The palette is earthy and artisan-inspired: olive green (#6b7c3a) as the primary brand colour, terracotta (#c87941) as the accent, and warm parchment (#f5f0e8) as the background. Typography uses the Outfit sans-serif font for UI elements and the Lora serif font for display headings and prices, creating a premium artisan aesthetic. The system defines reusable components including cards, buttons (primary, outline, success, danger, with size variants), form controls with focus rings, a sticky navbar with cart badge, product grid with hover animations, a toast notification system, a loading spinner overlay, and fully responsive breakpoints at 900px and 600px.

5. CONCLUSION

BharatBazzar represents a complete, production-quality implementation of a domain-specific e-commerce platform built around relational database management system principles. The project successfully demonstrates how a carefully normalised database schema, combined with advanced SQL constructs such as stored procedures, triggers, and views, can form the backbone of a robust and integrity-preserving application.

The database design achieves Third Normal Form across all twelve tables, eliminating data redundancy and update anomalies. The price-snapshot architecture in `Order_Items` ensures that the historical record of every transaction is immutable and accurate, reflecting the commercial reality of changing product prices. The transactional stored procedures `PlaceOrder` and `ProcessPayment` guarantee that the multi-step checkout process either completes entirely or rolls back completely, protecting the system from inconsistent intermediate states. The trigger-based inventory management provides a database-layer safety net that cannot be bypassed by any application-layer code.

The API layer demonstrates secure, stateless authentication using JWT and bcrypt, a clean RESTful design with appropriate HTTP methods and status codes, and efficient database interaction using parameterised queries through a connection pool. The frontend demonstrates a cohesive, accessible, and responsive user interface without relying on any external JavaScript framework, proving that vanilla web technologies are fully capable of producing polished, production-grade user experiences.

The project covers a breadth of DBMS course concepts: entity-relationship modelling, relational schema derivation, normalisation theory, constraint types (primary key, foreign key, unique, check), transactions and ACID properties, trigger design, stored procedure development, view creation, and index usage through foreign key constraints.

Future enhancements to BharatBazaar could include: full-text search indexing on product names and descriptions for faster keyword search, an admin dashboard frontend for product and order management, a review moderation workflow, support for artisan seller accounts with their own dashboards, integration with a real payment gateway, and image upload functionality with cloud storage integration.

--- End of Report ---